

Yume 夢

A Japanese-learning game where the only way to change the world in front of you is to say it in Japanese — built on a live, steerable video model.

Ronit Rashmikara · deadline 1 October 2026 · repository [ronithrashmikara/yume](#)

1. The idea in one paragraph

Most language apps test **recognition** — pick the right answer from four. Yume tests **expression**, and then shows you the consequence. You are dropped into a living, AI-generated world. It narrates itself to you in Japanese. When you say something back and the meaning lands, the live video steers to match within seconds. When it does not land, nothing moves. The video itself is the feedback.

The name comes from *kotodama* (言霊), the old belief that spoken words carry the power to shape reality. That metaphor is the product.

2. Why this needs a live model

This is the highest-weighted judging criterion, so it is worth stating precisely. Two properties make pre-rendered video impossible here:

- **The next sentence is unpredictable.** A learner can say anything, at any level, in any order. There is no finite set of clips to pre-generate.
- **The world keeps moving when you don't.** Every ~22 seconds the scene changes on its own, and every third change asks something of you. A clip cannot keep going while you hesitate.

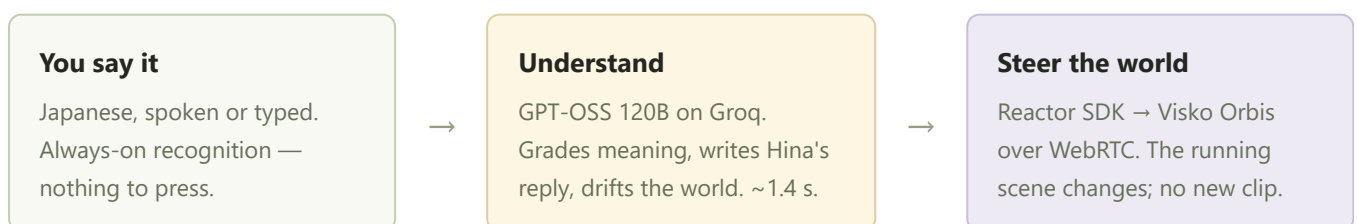
The second property is the stronger argument, and it is also nearly free: Orbis bills for every wall-clock second the GPU is held whether the scene changes or not. A static scene is wasted spend. An evolving one costs the same.

3. What is built today

AREA	WHAT IT DOES	STATUS
Core loop	Live Orbis video; Japanese input graded on meaning; correct answers steer the running scene via <code>set_prompt</code>	WORKING

Five ways to play	Park, Classroom, Night City (scripted objective chains), free-form world creation from an English idea, and companion mode	WORKING
Difficulty ladder	Four levels (single word → phrase → sentence → description) scaling both narration complexity and the grading bar	WORKING
Living world	Ambient drift every ~22s; every third beat is a "stakes" beat needing a spoken response; can be paused	WORKING
Hina (companion)	Bilingual companion; Groq writes her reply, Fish Audio speaks it, her reply steers the video	NEEDS VISUAL CHECK
Narration	Scene described in Japanese, pre-segmented server-side into hoverable words with reading + gloss	WORKING
Vocabulary + SRS	Double-click any word to save; Leitner ladder returns it after 1/2/4/8/16 days; a miss drops it to box 1	WORKING
Always-on speech	Mic arms itself when a world starts; live interim transcript; auto-sends after 1.1s of silence; deaf while the app itself is speaking	WORKING
Story log	End-of-run replay distinguishing what you changed from what the world did	WORKING
Site & polish	Landing page, fal-generated art, real sponsor logos, reduced-motion + focus states, mobile breakpoints; assets cut 5.9 MB → 1.8 MB	WORKING
Recording kit	Free preflight check, in-HUD live cost meter, one-command screen+mic recorder, shot-by-shot runsheet	WORKING
Demo video	Not yet recorded; landing page still shows a placeholder	NOT STARTED

4. How it fits together



Supporting layers

- **Fish Audio** — speaks narration and Hina, in Japanese and English.

- **Next.js 16 / React 19 / TypeScript** — app shell and five API routes (`check-answer` , `narrate` , `drift` , `companion` , `tts`).
- **localStorage** — saved vocabulary and SRS schedule. No account, no database.
- **fal.ai** — all backgrounds, the favicon, the mascot, and the presentation artwork. Not the live-video engine.

Two design decisions worth defending

The scene is an event log, not a growing string

Each change is recorded as an event with its source. One structure produces both the Orbis prompt and the end-of-run story, and it caps how many recent beats reach the prompt — without that cap, a world drifting every 22 s would grow the prompt without bound and the earliest details would compete with the newest for the model's attention.

Narration arrives pre-segmented from the server

Japanese has no spaces, so making every word individually hoverable normally means shipping a morphological analyser (~15 MB) to the browser. Instead the model returns the sentence already split into units with readings and glosses, and the concatenation is validated against the original.

5. What is verified, and what is not

Stated honestly, because the difference matters for the submission.

CLAIM	EVIDENCE	STATUS
Reactor key provisions a real GPU	Live session reached ACTIVE in ~3 s, then terminated cleanly	MEASURED
Orbis renders and steers in-browser	Screenshot of a cat rendered in the Park scenario after a graded turn	OBSERVED
Grading works end to end	Level-1 single word ねこ accepted; full sentence corrected naturally; ~1 s round trip	MEASURED
Narration segments correctly	9 tokens returned, concatenation matched the sentence exactly	MEASURED
Hina steers the world	4 / 4 test turns produced a scene change; ~1.4 s average	MEASURED
Ambient + stakes beats generate	Ambient 2.6 s cold, stakes beat 0.8 s, both usable	MEASURED
SRS schedule behaves	New word due immediately; correct answers climb 2 → 4 → 8 d; a miss returns it next day	MEASURED
Hina stays visually consistent	Reference-frame conditioning + locked text description are implemented, but nobody has watched her across several steers	UNVERIFIED
Steering latency (answer → visible change)	Never timed against a stopwatch	UNVERIFIED

6. What it costs to run

Rates read live from Reactor's pricing API, not estimated.

ITEM	RATE	NOTES
Visko Orbis Stable	\$0.0097 / second	= \$0.582 / minute
A 2-minute demo take	≈ \$1.16	Rehearsal ≈ \$0.29

Worst case, one orphaned session	\$2.91	Token caps sessions at 300 s
\$250 of credit	≈ 429 minutes	≈ 214 full 2-minute takes
Groq GPT-OSS 120B	\$0.15 / \$0.60 per 1M	Fractions of a cent per turn
Verification spend to date	\$0.79	Two live session tests

Three cost traps

Idle bills at full rate. Reactor meters every wall-clock second the GPU is held, "even if you are idle". **Billing granularity is ambiguous** — docs say per session-minute, the pricing API quotes per-second; assume short aborted takes round up. **Sessions hard-stop at 300 s**, so a take running past 5:00 loses the world mid-shot. The in-HUD meter turns amber at 4:00 for exactly this reason.

7. What we could build next

Ordered by judging impact per hour of work. The real-time story is already strong, so the marginal value of more mechanics is lower than the marginal value of a convincing demo — but there is room for both.

Barge-in · ~2 h

Interrupt Hina mid-sentence and she stops and listens. We already track when she is speaking, so the machinery exists. This is the single change that most makes her feel alive rather than turn-based.

Silence she fills · ~2 h

Say nothing for ~15 s and she prompts you — どうしたの? — or points something out. A conversation that survives hesitation reads as genuinely live on camera.

Listening comprehension · ~3 h

She asks you to do something and the world only moves if you understood *her*. The app currently tests production only; this adds the input half, which is what actually builds fluency.

Pronunciation feedback · ~3 h

Web Speech returns a confidence score per result. Low confidence on a word you meant to say is a mispronunciation signal she can gently echo back. Genuinely hard for a textbook to do.

Two more worlds · ~2 h

Cheap and safe. Makes the app feel fuller on camera and demonstrates that the scenario format generalises.

Shareable story card · ~2 h

Render the end-of-run story log as an image. Turns a session into something a learner would post, which is the cheapest distribution a language app can have.

8. Risks and open questions

Hina's visual consistency is unproven

The headline feature rests on Orbis honouring a reference frame across repeated `set_prompt` calls. Both mitigations are implemented (image conditioning + the locked description restated every steer), but if she visibly becomes a different person after a few turns, the companion demo needs rethinking. **Two minutes of browser testing resolves this.**

The repository is private

Judges cannot read the code. Git history has been scanned and contains no committed secrets, so it is safe to publish — but it must actually be flipped before submission.

Groq free-tier limits during a long demo

Living-world drift plus narration is roughly 5–6 calls per minute. Comfortable against 30 RPM, but a long unattended session could approach the 1,000/day cap.

Everything is single-player and local

Vocabulary lives in `localStorage`: per-browser, no account, no sync. Correct for a hackathon prototype, and worth saying out loud rather than implying a backend exists.

9. Before submission

- Verify Hina stays visually consistent across several steers **BLOCKING**
- Rehearsal take (~\$0.29) to calibrate steering latency, then record the real 2-minute demo
- Embed the video on the landing page, replacing the placeholder
- Make the repository public **BLOCKING**
- Complete the submission form; verify every link after pasting
- Confirm no API key is visible in any recorded frame